Afterimage: Post-Revocation Leakage and Existence Inference in Multi-Tenant Retrieval-Augmented Generation

Mohit Talgotra*, Ayush Amit Govindwar*, Vipin Chandran*, and Sivaranjani A
School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India
{mohit.talgotra2023, ayushamit.govindwar2023, vipin.chandran2023}@vitstudent.ac.in, sivaranjani.a@vit.ac.in
*Equal contribution.

Abstract—Multi-tenant retrieval-augmented generation (RAG) applications copy access-control state and document content into derived artifacts: ACL copies in the vector index, semantic-cache entries, and chat memory with model-written summaries. Revoking a user's access in the identity system does not revoke these copies. We measure what they disclose after revocation, separating secrets the user had already been shown (re-disclosure, a violation of complete mediation) from secrets the user had never seen. On real Enron e-mail with Llama-3.1-8B (4-bit, served locally), [first result sentence: leakage and new disclosure under baseline, and through which artifact, from p_min_enron]. A published retrieval-time enforcement design removes the index channel and leaves [Y] of the leakage. We then compare read-time defenses for derived state, provenance taint and a per-user permission epoch: [defense result from p_min_enron: whether each reaches zero leakage and what it costs the revoked user on documents it still holds]. In an unmodified LangChain, Chroma and RedisVL deployment, a per-user semantic cache and agent memory both disclosed revoked e-mail although retrieval refused it. A user who pastes back what they once received defeats every artifact-level defense.

Index Terms—Access control, information-flow control, large language models, membership inference, multi-tenancy, provenance, retrieval-augmented generation, revocation, side channels.

I. Introduction

Retrieval-augmented generation [1] lets a language model answer from a document collection it was not trained on. In enterprise deployments the collection is shared by several tenants and roles, so the application decides per request which documents a user may see. The common design copies document ACLs from the identity system into the vector index and filters retrieval on them [2]–[5].

That design makes the identity system the source of truth and every other component a copy, and a RAG application makes several copies that outlive the request that created them. The index holds materialized ACLs, refreshed by a crawl or a webhook [6]. A semantic cache returns stored answers to similar questions [7], [8]. Chat memory keeps earlier turns and, after a few turns, a summary written by the model, which names no source document [9]–[11]. When an administrator revokes access, the identity system changes at once; each copy changes only when something propagates the revocation to it. We call what the copies serve in the meantime post-revocation leakage.

a) Why it matters if the user saw it before.: A revoked user has, by definition, been allowed to read the document, so an obvious objection is that re-showing it reveals nothing. We take the objection seriously and measure around it. First, we separate re-disclosure (secrets the user was shown before revocation) from new disclosure (secrets of the document the user was never shown). Only the second is new information; the first is still a failure of complete mediation [12], which access-control standards require to be closed promptly when a user changes role or leaves [13], and which matters for users who no longer hold their copies. Second, derived state is shared: a cache entry or a laundered answer can reach a colleague who never had access. Third, a user who changes department (a mover) keeps an active account, so the window is open to an ordinary, authenticated client.

b) Existence inference.: A separate question is whether the application reveals that a restricted document exists without returning its content. Relevance scores, reranker confidence, latency and refusal wording can all depend on a hidden document [14], [15], and caches and LLM serving have their own timing channels [16]–[19]. We measure existence inference as a distinguishing game between two worlds that differ only in the target document, for insiders (same tenant) and outsiders, and keep it separate from leakage throughout.

c) Research questions.:

RQ1 Which derived artifacts disclose a revoked document, and how much of what they disclose is new to the user?

RQ2 How long does each leak last in the day after revocation, under a daily ACL crawl and default cache and session settings?

RQ3 What can an insider or outsider infer from response metadata about documents they cannot read?

RQ4 Which mitigations close each channel? How do read-time defenses for derived state (provenance taint, permission epochs, session flush) and published retrieval-time designs compare?

RQ5 What do the defenses cost the revoked user on documents it still holds, and in authorization load?

d) Contributions.:

1) A measurement procedure for post-revocation leakage per derived artifact, with paired revocation events, right-censored lifetimes, read-only probes that do not refresh what they measure, and a decomposition into re-disclosure and new disclosure (Section VI).
2) Measurements on real e-mail (Enron [20], [21]) with Llama-3.1-8B (4-bit, served locally), whose chat memory is summarized by the same model, and a check of the two main leaks in an unmodified open-source stack (Section VIII).
3) A twin-world measurement of existence inference that separates insiders from outsiders and content channels from metadata channels, with intervals clustered by target document [22], run with a deterministic stub generator (Section VIII-C).
4) A comparison of read-time defenses for the derived state of a RAG pipeline (provenance taint and a per-user permission epoch) against a published retrieval-time design, on leakage, utility for the revoked user, and authorization load, with session flush and further designs in the stub harness (Sections IV, VIII-D and VIII-E). The mechanisms are known (information-flow control [23]–[25], complete mediation); the contribution is their instantiation for LLM-derived artifacts and the measured trade-off between them.

e) Summary of findings.: [Fill after the runs of configs/min/, at most five numbers: LM and LM_new under baseline (p_min_enron), the retrieval-time design's residual leakage, the retained-document recall of epoch against full, insider existence AUC under baseline and under the defense (stub).]

f) Scope.: The deployment model is grounded in documented framework and product behaviour (Table II), but every measurement comes from our testbed rather than from a production system, apart from the real-stack check (?? VIII-F0b). We use one 4-bit 8B model and 50 revocation events per cell. Zero measured leakage means zero under a corpus-unique exact-match judge and an unvalidated model judge (Section VI), for an adversary who queries the system. Section X states where that does not hold.

II. Related Work

We organize prior work around the paper's three claims: that derived state outlives authorization changes, that response metadata reveals hidden documents, and that provenance-based read-time enforcement closes the first without opening the second. Table I positions the closest work on the dimensions that matter here.

A. Derived state outlives authorization changes

a) Access control for search and retrieval.: Document-level security in enterprise search predates RAG. Bailey et al. compare copying permissions into the index with checking each result at query time [36]; fine-grained access control by query rewriting is the database analogue [37]. RAG systems filter retrieval with permissions derived from an IAM system [2], for sensitive patient data [3], or with dynamically partitioned vector indexes for role-based filtering [4]; products expose the same pattern as metadata filters [5], [38]–[40]. Filtered nearest-neighbor search treats the pre- versus post-filter choice as an efficiency problem [41], [42]; we show it is also a security choice. A materialized ACL copy is a materialized view, stale for as long as its maintenance lags [43]. None of these designs addresses artifacts derived before a permission changed.

b) Revocation and stale authorization.: Complete mediation requires every access to be checked against current authority [12]. Revocation is an old problem for capabilities, where selective revocation needs indirection [44], and for cryptographic file systems, where lazy revocation defers re-encryption until the next write [45], [46]; our read-time check is the opposite trade-off, paying on every read so that no stale copy is served. Secondary and approximate authorization models cache and infer decisions for availability and must bound how stale they are [33], [34]. Zanzibar binds checks to consistency tokens to avoid the "new enemy" problem [26]; our artifacts are derived before the revocation and re-served after it with no intervening write. Measurement work on certificate revocation [35] is the methodological template for measuring how long a revoked credential stays usable; we apply that view to RAG artifacts.

c) Deletion, unlearning and derived data.: A system must forget data together with everything computed from it [47], formally captured by deletion-compliance definitions [48]. DELF makes cascading deletion first-class at Facebook and reports that developers routinely omit derived-data deletions: before enforcement they handled deletion for only 20% of new edge types [28]. K9db and Edna build compliant deletion and disguising into web-application storage [29], [49], and GDPR compliance has measurable cost in databases [50]. Unlearning removes data from trained models [51] and, for RAG, from the knowledge base [52]; Tavakoli and Sanderson audit whether RAG systems forget deleted material [30]. Revocation differs from deletion in the way that matters for derived state: the document stays available to other principals, so a derived copy must be filtered per principal on every read rather than removed.

d) Chat memory.: LLM applications keep conversation state as recent turns plus a rolling summary written by the model [9], [10], [53], or as managed long-term memory [11], [54], [55]. Memory can be extracted by a crafted query [32]. No prior work we found asks what

TABLE I
Closest prior work. Covers: which derived artifacts the work addresses. Per-principal: removal for one user while others keep access (revocation), as opposed to removal for everyone (deletion). Generated: handles artifacts written by a generative model, which carry no source identifiers. Lifetimes: measures how long a stale copy stays servable.

Work	Index	Cache	Memory	Revocation	Per-principal	Generated	Lifetimes	Kind
Zanzibar [26]	–	authz cache	–	yes	yes	–	–	mechanism
Permission-aware RAG [2], HoneyBee [4]	yes	–	–	partly	yes	–	–	mechanism
Thoth [24], Qapla [27]	yes	–	–	–	yes	–	–	mechanism
DELF [28], K9db [29]	stores	stores	–	–	–	–	–	mechanism
Tavakoli and Sanderson [30]	yes	–	–	–	–	–	–	measurement
FIDES [25], CaMeL [31]	–	–	agent state	–	–	yes	–	mechanism
MEXTRA [32]	–	–	yes	–	–	yes	–	attack
SAAM [33], recycling [34]	–	authz cache	–	policy change	yes	–	–	mechanism
Liu et al. [35] (web PKI)	–	CRL/OCSP	–	yes	–	–	yes	measurement
This work	yes	yes	yes	yes	yes	yes	yes	measurement + comparison

memory serves after the user's permissions change, which is where our measurements find the longest-lived leaks.

B. Metadata reveals hidden documents

Statistics computed over documents a user cannot read leak information about them: under a system-wide index with post-filtering, an unprivileged user recovers collection statistics from relevance scores [14]. Cross-site search attacks infer the existence of private records from search timing [15], and web applications leak state through response size and timing [56]–[58]. In LLM serving, prompt and KV caches create timing channels [16]–[18], [59], speculative and efficient inference leaks through timing [19], token lengths leak encrypted responses [60], and semantic caches are vulnerable to key collisions [61], [62] and motivate user-scoped designs [63]. Membership inference against the RAG datastore uses the generated answer [64]–[67], and extraction attacks recover datastore content [68]–[70]; embeddings themselves can be inverted [71]. Defenses include differentially private retrieval [72] and synthetic datastores [73]. Those attacks target a datastore the model can retrieve from. Our attacker cannot retrieve the target at all and asks whether metadata or shared state betrays it; we report insiders and outsiders separately because shared caches give the two very different power, and we use the low-false-positive operating point of Carlini et al. [74].

C. Provenance-based read-time enforcement

Provenance-tainted revocation is dynamic information-flow tracking [23], [75]; operating systems [76], [77] and applications [78], [79] attach labels to data and propagate them through computation, and provenance-based access control names the model in which decisions depend on derivation history [80], [81]. Thoth enforces declarative policies on derived data inside a Lucene-based retrieval system, with a per-task taint set [24]; Qapla enforces policies on database queries without trusting the application [27]. For LLM systems, information-flow control now spans agent planners [25], [31], [82], [83], isolation architectures [84], contextual-integrity agents [85], access control over model parameters [86], [87], and permissive analyses that reduce over-tainting by asking which inputs actually influenced an output [88]; context attribution [89] is the corresponding tool. These works establish the mechanism. Two things differ here. Our unit of taint is the persistent artifact (a cache entry, a memory turn, a summary sentence) rather than a task or a plan, and our question is what the mechanism costs compared with coarser invalidation, measured on in-session utility. The usual limit of information-flow control, that a principal who read data can re-type it, also bounds our defense, and we measure it.

a) Search protocol.: On 14 October 2026 we ran 21 targeted queries through general web search, the OpenAlex and Crossref APIs, and publisher and proceedings pages (ACM DL, IEEE Xplore, USENIX, Springer, PMLR, NeurIPS). The queries covered permission-aware and multi-tenant RAG, RAG unlearning and deletion, stale authorization and revocation in distributed systems, information-flow control for LLM agents, chat-memory privacy, semantic-cache and KV-cache side channels, relevance-score leakage in multi-user search, backward-private searchable encryption and revocable encryption, and a direct sweep for post-revocation leakage in RAG. The search was targeted rather than systematic; work on agent memory, authorization caching, filtered vector search and semantic caches was added afterwards from the authors' knowledge of these areas. We found no peer-reviewed work that measures post-revocation leakage per derived artifact in a RAG pipeline.

III. Problem Setting and Threat Model

A. Pipeline and derived artifacts

A request from user u passes through a semantic-cache lookup, retrieval of the top-k chunks from a vector index, reranking, and generation. The application keeps per-session chat memory: recent turns verbatim and, past a length threshold, a rolling summary written by the generator. The identity system (IAM) is the source of truth. Three artifacts hold derived state that is not recomputed per request:

- the index ACLs, a copy of document permissions stored beside the embeddings and refreshed by a periodic crawl, by webhook, or both;

- the semantic cache, which returns a stored answer when a new question's embedding is within a similarity threshold, until a time-to-live (TTL) expires;
- the chat memory, whose summary condenses earlier answers and names no source document.

Table II grounds each parameter in documented framework and product behaviour.

B. Adversaries and goals

Revocation leakage. A legitimate user has used a document and then loses access to it. The adversary is that user acting as an ordinary authenticated client, or a colleague in the same tenant who never had access. Access is removed for one document (single-document), by removing a group (role), by a department transfer that removes one group and adds another (mover), or for the whole account (offboarding), or when an administrator narrows the document's ACL (reclassification), which removes it from every user of a group at once. A mover keeps an active session and account, which makes it the realistic case; offboarding assumes the deprovisioned user can still reach the application (for example until an SSO session or an API token expires) and is reported for completeness. The goal is to learn secrets of the revoked document that the user was never shown (new disclosure), including those of a version written after the revocation, or to be shown again what they were shown before (re-disclosure). Re-disclosure gives the user nothing new, but it violates complete mediation [12] and the obligation to cut off access on a role change [13], and it is the step that moves revoked content into state other users read.

Existence inference. An API client without access to a target document decides whether it exists in the collection. An insider belongs to the target's tenant; an outsider does not. The adversary sees what an API client sees: the response text, the latency and, depending on the exposure profile, cited sources with relevance scores and the reranker confidence. It knows the target's topic and a question about it (the membership-inference setting [91]); discovering unknown topics is out of scope.

C. Threats

Table III lists the leak mechanisms and the switch that addresses each.

T1–T3 are revocation threats, evaluated by the revocation experiment. T4–T7 are existence channels, evaluated by the twin-world experiment; T2 also carries existence signal when a cache is shared between users. T8 does not arise from a copy the application made but from the user re-supplying content, which is why assumption A1 (Section IV-D) states it; measuring it is future work.

IV. Read-Time Defenses for Derived State

All defenses below share the same retrieval and response-shape settings: a live IAM check on every retrieved chunk, ACL filtering inside the vector query, and constant-shape responses. They differ in how they decide whether a cache entry or a memory item may still be served.

A. Provenance-tainted revocation

a) Taint.: Every derived artifact a carries a taint $T(a) \subseteq \mathcal{D}$, the set of source documents whose content it may contain. A cache entry has the taint of the answer it stores. An answer has the union of the taints of the chunks and memory items in its prompt; a summary has the union of the taints of the turns it replaces:

$$T(\text{answer}) = \bigcup_{c \in \text{context}} T(c), \qquad T(\text{summary}) = \bigcup_{t \in \text{turns}} T(t). \tag{1}$$

The first equation makes taint transitive through memory: an answer built from an earlier answer about document d inherits d.

b) Enforcement.: Eagerly, when a revocation event fires, tainted cache entries are evicted and tainted memory items are purged. Lazily, on every read, artifact a is served to user u at time t only if

$$\forall d \in T(a) : \ \text{IAM}(u, d, t) = \text{allow}. \tag{2}$$

The lazy check refers to the identity system at read time, so it does not depend on whether a revocation event reached the artifact. The cache evaluates it only for candidates above the similarity threshold, from the most similar down, so authorization work grows with the number of servable candidates rather than with the cache size.

c) Sentence-level taint.: A summary's taint is the union of everything it condenses, so one revoked source discards the whole summary, including material from documents the user still holds. With an extractive summarizer, each summary sentence can carry the taint of the turn it came from, and the lazy check then drops only the tainted sentences (full_fine). This stays sound with any generator, since a filtered summary's taint is the union of the sentences it keeps. Two things limit it. With model-written summaries there is no sentence-to-source mapping, so full_fine reduces to full; this paper does not measure it. And an answer's taint covers every memory item in its prompt unless the generator reports which items it used: our stub does, real models do not, so with real models an answer given after the user discussed the revoked document carries that document's taint even if it did not use it. Context attribution [89] or permissive information-flow analysis [88] is the route to finer answer taint, at the price of a soundness argument we do not make.

B. Simple alternatives

A reviewer's first question is why taint is needed at all. We implement the two obvious alternatives with identical retrieval and response settings.

TABLE II
Grounding of the deployment model in documented behaviour (documentation accessed October 2026). The baseline preset uses the documented defaults; settings that deviate are named presets of their own.

Component	Documented behaviour	Our setting
Index ACLs	Security trimming stores user or group identifiers in an index field and filters on them at query time [5]; vector stores filter on payload or metadata fields [38]–[40]; connectors ingest ACLs with items and re-crawl on a schedule [6].	Per-user ACL booleans in Chroma metadata, pre-filter in the query; sync by daily crawl (the testbed also implements hourly crawls and webhooks with loss and a backstop).
Semantic cache	Similarity threshold and TTL; scoping by user or other fields is configured by the application [7], [8], [63], [90].	Threshold 0.90, TTL 3600 s; per-user scope in baseline, tenant-shared scope as the named misconfiguration baseline_shared.
Chat memory	Recent turns kept verbatim, older turns replaced by a model-written running summary [9], [10], [53].	Summary after 8 turns, 4 kept verbatim, written by the generator model; sessions retained for the whole run.
Revocation	Access must be modified or disabled when users change role or leave [13].	Single document, role removal, department move (mover), offboarding, document reclassification.

TABLE III
Threats and the mitigation switch addressing each.

	Mechanism	Mitigation
T1	Index ACL copy is stale	live IAM check
T2	Cached answer outlives permission, or is shared across users	per-user scope; eviction, lazy taint or epoch check
T3	Memory and summaries retain content	taint purge and lazy check; epoch check; flush
T4	Retrieval score of a withheld candidate	ACL pre-filter; scores over authorized chunks; score bands
T5	Reranker confidence	as T4
T6	Latency differs between paths	padding, deadline, no fast path
T7	Refusal wording differs	uniform refusal text
T8	A revoked user re-enters content received earlier, which enters shared state	query fingerprinting

- Permission epoch (epoch). Each user has an epoch in the IAM that every revocation increments; a reclassification increments the epoch of every user who loses the document. Cache entries and memory turns carry the epoch of the user who created them and are dropped on read when it is stale; a summary written after a revocation does not absorb stale turns. This is a read-time check, so it covers lost events, but it discards everything the user derived before any revocation, whatever its sources.
- Session flush (flush). On a revocation event, every session and cache entry of that user is dropped. It is eager and depends on event delivery: a revocation hook that is never delivered, as happens with webhooks or a message bus, leaves the user's state in place.

The read-time alternatives reach zero leakage by construction for the artifacts they cover. The comparison that matters is therefore what each one costs (Section VIII-E). The cost differs most when a document is reclassified: the epoch of every user who loses it is bumped and all their sessions are discarded, whereas taint discards only what derives from that document. The existence experiment also runs full with a per-user cache (full_user), so that the comparison does not mix the defense with cache scope.

C. Constant-shape responses

Responses expose relevance scores only in coarse bands and only over chunks the user may see, use one refusal text for every reason, and pad latency up to the next multiple of a bucket (500 ms). Padding is applied after all computation.

D. Security argument and assumptions

Invariant 1. For every derived artifact a (cache entry, memory turn, summary or summary sentence), if content of a restricted document d reached a through retrieval or memory, then $d \in T(a)$.

Proposition 1. Assume A1–A4 below. If the invariant holds and every read of a derived artifact applies (2), then after u loses access to d no response to u is generated from, or re-serves, an artifact containing content of d, and no retrieval for u returns a chunk of d.

Proof sketch. Content enters an artifact only through a generation step, whose inputs are the retrieved chunks, the memory items in the prompt and the query. The pipeline writes the union of the chunks' documents and the items' taints as the response taint, cache and memory writes store it unchanged, a cache hit re-serves an entry with its stored taint, and a summary's taint (or each summary sentence's) is the union of the taints it condenses. By induction over the sequence of writes, the invariant holds for every artifact. After revocation $\mathrm{IAM}(u, d, t) = \text{deny}$ for every read (A4), so any artifact with $d \in T(a)$ fails (2), and the live check removes d's chunks from retrieval. The response is then generated only from artifacts and chunks free of d's content (A1–A3). □

A randomized test runs sessions with frequent summarization, at turn and sentence granularity, and checks that no artifact holds a secret unique to a restricted document

without that document in its taint; it also confirms that dropping transitivity violates the invariant.

a) Assumptions.:

A1 User input is untainted. Pasted restricted content is not attributed to its source. An optional switch fingerprints queries against restricted secrets and taints a query containing one verbatim; paraphrased re-entry is the declassification limit of information-flow control [23], [75]. Measuring this channel is future work (Section XI).

A2 No untracked inputs. Tools, web search or plug-ins would have to label their output.

A3 Document ACLs are correct. A document that copies restricted figures under a broader ACL legitimately serves them.

A4 Consistent authorization reads. Checks read IAM state at least as new as the revocation; a lagging replica reopens the "new enemy" window [26].

b) Presets.: Table IV defines the deployments. strawman is a deliberately leaky reference. baseline follows the documented defaults of Table II. baseline_shared is the same with a tenant-shared cache, a misconfiguration we measure separately. legacy purges memory by document identifier, which cannot match a summary. authz_prefilter and authz_postfilter implement the enforcement patterns of permission-aware RAG [2] and Zanzibar-style check-after-retrieve [26] on our switches; they are reimplementations of the pattern, not those systems.

V. Corpora and Systems

a) Primary corpus: Enron e-mail.: All results in the main text use Enron e-mail [20] with the questions and gold answers of the EnronQA benchmark [21]: real e-mail, questions and answers; tenancy and permissions derived from each message's recipients; a canary sentence injected into every restricted e-mail.[1] Ten inboxes, sampled from the 110 that have enough mail, form five tenants of two inboxes each, and each inbox owner has an assistant. Mail addressed to few people is restricted to its inbox group and mass mail is internal to the tenant, giving 650 documents of which 500 are restricted (1.7 chunks each at 300 tokens). E-mails in one thread form a project. Facts are short spans of the gold answers that occur verbatim in the e-mail and not in the question; 43% of them also occur in another e-mail of the corpus, so the judge scores only the corpus-unique ones (Section VI). Following the secret-sharer method [92], the canary sentence of each restricted e-mail is unique, which gives a zero-ambiguity leak signal on real text. The text, questions and answers are real; the tenancy and ACLs are ours.

b) Generated corpus: a controlled stub harness.: A template-generated corpus provides exact ground truth and controlled variation; it backs the test suite and the pilot study (Section A) and is not used for the results in the main text. Each seed generates its own corpus, so seeds are independent replicates rather than re-draws of events from one corpus. Every corpus has five tenants of 100 documents (40 public, 30 internal, 30 restricted) and eight users per tenant (six employees, two contractors). Restricted documents come in projects of two, so the twin world without a target still holds a same-project document, and carry six to twelve filler paragraphs drawn from sixteen boilerplate memo templates between their fact paragraphs (about 420 words, 2.4 chunks each), so one document's secrets span several chunks. Numeric fact values are unique across the corpus, every restricted document carries a canary, and the training tenants (t1–t3) and test tenants (t4–t5) use disjoint document templates.

c) Retrieval stack.: Embeddings come from all-MiniLM-L6-v2 [93], [94], reranking from the ms-marco-MiniLM-L-6-v2 cross-encoder, and storage from ChromaDB. Retrieval takes the top 5 chunks and passes 3 to the generator, with a minimum cosine relevance of 0.25. The semantic cache uses a threshold of 0.90 and a TTL of 3600 s unless varied. Memory writes a summary after 8 turns and keeps 4 turns verbatim.

d) Generators.: The model run uses Llama-3.1-8B (instruction-tuned; 4-bit, the default build of Ollama, served locally) at temperature 0. The same model writes the memory summaries, with a running-summary prompt in the style of LangChain's summary memory [9]. A deterministic stub that answers extractively from the best-matching context unit, with an extractive summarizer that keeps sentences carrying figures, serves as a validation harness and runs the broader defense comparison, the existence experiment and the post-revocation update experiment; stub and model results are never pooled.

e) Clock.: A simulated clock adds virtual waits (time between probes, padding) to real compute time, so one run covers up to 48 h after revocation. The existence experiment uses the stub, whose generation latency is simulated on the clock; its timing channel therefore reflects the testbed's compute time and the stub's latency model, not a served model.

VI. Measurement Methodology

A. Revocation experiment

Each revocation event proceeds as follows.

1) Sample a restricted target document with at least one scorable secret, and a victim who may read it.
2) Warm-up. In one chat session the victim asks a question about the target, then about four other documents it may read (public or internal fillers, plus two restricted documents of the target's project or group where the experiment calls for it), then a second question about the target. This leaves copies in the cache, in memory and in a summary.
3) Wait for the revocation gap g, drawn uniformly from $[0, 600]$ s, then revoke the victim's access to the

[1] https://huggingface.co/datasets/MichaelR207/enron_qa_0922.

TABLE IV
Deployment presets. "Stale ACL" means the index ACL copy is trusted between syncs. All presets except strawman retrieve within the user's tenant and expose scores computed over chunks the user may see.

Preset	Retrieval	Cache	Memory	Response metadata
strawman	all tenants, post-filter, withheld scores exposed	global, no hook	no purge	raw scores, distinct refusals
baseline	pre-filter, stale ACL	per user, no hook	no purge	raw
baseline_shared	pre-filter, stale ACL	tenant-shared, no check	no purge	raw
legacy	pre-filter, stale ACL	tenant, evict on revoke	purge by document ID	bands, uniform refusal, 30 ms floor, fast refusal
authz_prefilter	pre-filter, live IAM	per user, no hook	no purge	raw
authz_postfilter	post-filter, live IAM	per user, no hook	no purge	raw
epoch	pre-filter, live IAM	per user, epoch check	epoch check	bands, uniform refusal, 500 ms buckets
flush	pre-filter, live IAM	per user, flush on revoke	flush on revoke	as epoch
full	pre-filter, live IAM	tenant, evict + lazy taint check	transitive taint purge + lazy check	as epoch
full_user	as full	per user, evict + lazy taint check	as full	as epoch
full_fine	as full	as full	as full, sentence-level summary taint	as epoch

target (single-document revocation; the testbed also implements role, mover, offboarding and reclassification, which this paper does not measure). The gap is the time since the victim last used the document; leakage is conditional on it, because cache entries expire.

4) In the update experiment only, edit the target right after revocation: every numeric fact value changes and a new canary replaces the old one, and the new version is re-indexed under its (stale) ACL.
5) Probe at scheduled times (0 s, 1 h and 24 h in the model run; 0 s to 48 h in the stub runs). Each probe time asks a same-session "remind me what you told me earlier" question (memory, cache, index), a fresh-session question about the target (cache, index), a fresh-session question about a target fact the warm-up never asked about (rotating), and fresh-session questions about every other warmed-up document the victim also lost. In every run, fresh-session probes repeat the warm-up question verbatim (there is no paraphrase bank), so cache hits there are not controlled by the cache threshold.

Probes are read-only: they neither write the cache nor append to memory. Otherwise every "remind me" probe would store the leaked answer again and the measured lifetime of memory and cache leaks would partly be caused by the measurement. Event sampling is seeded by (seed, event), so every configuration sees the same targets and victims and comparisons are paired.

a) The judge.: A secret is a fact value of the target or its canary. A fact value counts only if it is corpus-unique: it occurs in no other document's title or text. Values such as a city name or a hazard ratio of 0.58 recur across documents (21% of fact values in a corpus without the uniqueness constraint, 43% on Enron), and a response containing them proves nothing about where its content came from. A secret is disclosed when its exact value appears in a post-revocation response. An independent model judge (qwen2.5:7b) is asked, for every scorable fact the exact match missed, whether the response conveys it in other words. The model judge is not validated against human labels; we report what it adds as an indication only. Exact matching is a lower bound on disclosure.

b) Leakage metrics.: Let S_e be the scorable secrets of the target in event e, $W_e \subseteq S_e$ those the victim was shown during warm-up, and $L_e \subseteq S_e$ those disclosed after revocation. Then

$$\mathrm{LM}_e = \frac{|L_e|}{|S_e|}, \quad \mathrm{LM}_e^{\mathrm{new}} = \frac{|L_e \setminus W_e|}{|S_e|}, \quad \mathrm{LM}_e^{\mathrm{re}} = \frac{|L_e \cap W_e|}{|S_e|}, \tag{3}$$

and LM is the mean over events. $\mathrm{LM}^{\mathrm{new}}$ is information the victim did not have; $\mathrm{LM}^{\mathrm{re}}$ is the complete-mediation violation. A per-user cache and a memory that stores questions and answers hold only what the victim was shown, so their $\mathrm{LM}^{\mathrm{new}}$ is zero by construction; new disclosure then comes from the stale index or from another user's cache entries. Agent memory that keeps tool outputs can also disclose content the user never saw, since it holds the retrieved text itself (?? VIII-F0b). With a post-revocation edit, $\mathrm{LM}_e^{\mathrm{upd}}$ is the fraction of the new version's secrets disclosed, which is new by construction. We also report the leak rate (events with $\mathrm{LM}_e > 0$), the canary-only leak rate (a judge-free check), the per-probe leak probability, LM restricted to probes at least 1 h and 24 h after revocation (an attacker who does not race the revocation), per-artifact $\mathrm{LM}_{\mathrm{cache}}, \mathrm{LM}_{\mathrm{index}}, \mathrm{LM}_{\mathrm{memory}}$ (attributed from ground truth; not exclusive, so they do not sum), and the blast radius $\mathrm{LM}_{\mathrm{all\ lost}}$ over every warmed-up document the victim lost. When memory is summarized we report the share of the target's secrets that survive into any summary.

c) Lifetimes.: An event's lifetime is the first probe time after which leakage stops for good; events still leaking at the last probe (24 h in the model run, 48 h in the stub runs) are right-censored and summarized with the Kaplan–Meier estimator [95]. Lifetimes resolve to the probe grid. For a periodic crawl with period Δ and a revocation at a uniform phase, remaining staleness is uniform on $[0, \Delta]$ with median $\Delta/2$, and cache leaks end at the TTL; these closed forms validate the simulator and are reported as such (Section B), not as findings. Chat memory has no closed form: its lifetime is set by session retention, which this paper does not vary (sessions are kept for the whole run).

B. Existence experiment (twin worlds)

For each restricted target the testbed builds two worlds that differ only in the target: world 1 contains it, world 0 does not. Both worlds share every random draw. With probability 0.5 a legitimate user queries the target first. The attacker (an insider and, separately, an outsider) then sends four probes in each world: a direct question, an overview request, a comparison with a public topic, and a bare topic string. The label is the world.

a) Attackers.: The existence experiment uses the stub generator. Attackers are trained on three shadow tenants (e1–e3) and tested on two held-out tenants (e4–e5). The main attacker is a gradient-boosting classifier [96] trained separately per channel: score (confidence, number of sources, top source score), reranker, latency (rounded to 1 ms), refusal (refusal and "denied" words, response length), content (query–answer overlap, digit count) and combined. Logistic regression and a rule-based attacker are also run. Exposure profiles control what the attacker sees: text and latency, plus sources and scores, or plus reranker confidence.

b) Metrics.: We report AUC per probe and per trial (the four probes of a target averaged), TPR at 1% FPR, and balanced accuracy, for insiders and outsiders separately and for targets with and without a same-project sibling. Where the ACL pre-filter hides the target from the attacker in both worlds, the metadata channels are equal by construction and only shared state can differ; we mark those cells as structural rather than empirical.

C. Defense comparison and cost

a) Collateral cost.: After revocation the victim is also asked about every warmed-up document it still holds, once in the original session and once in a fresh session. Recall and refusals on these probes measure what each defense discards that the user is still entitled to.

b) Authorization load.: We report IAM calls and per-document authorization decisions per query, taken from the revocation runs.

D. Statistics

Events that share a target document, and probes that share a target, are not independent. All intervals are 95% percentile bootstraps that resample target documents as clusters (on the generated corpus of the pilot a cluster is a (seed, document) pair) (2000 resamples; 1000 for AUC, which follows the clustered-ROC setting of Obuchowski [22]). Paired differences ΔLM between configurations are computed on identical (seed, event) pairs; we report the target-clustered interval and a Wilcoxon signed-rank test on per-target mean differences, and adjust the p-values of each table with Holm's method [97]. A difference between an exact zero and a non-zero cell is reported without a test.

VII. Experimental Setup

Table V lists the experiments; each row is one configuration file in configs/min/ of the released repository. All use the Enron corpus with canaries and five seeds, which re-draw revocation events from the one corpus. The model run has 50 revocation events per cell (5 seeds × 10); the stub runs have 200.

a) Probe schedule.: The model run probes 0 s, 1 h and 24 h after revocation; the stub runs probe at 0 s, 10 min, 1 h, 4 h, 12 h, 24 h and 48 h.

b) Environment.: One laptop with an RTX 3050 laptop GPU (6 GB) running Ollama [version] with the default 4-bit build of Llama-3.1-8B; Python 3.14, scikit-learn 1.9.1, ChromaDB 1.5.9, sentence-transformers 6.1.0. Runs are sequential. Latency results are comparable within the paper only.

VIII. Results

Unless stated otherwise, results are for Llama-3.1-8B (4-bit, served locally) with model-written summaries on Enron, 50 revocation events per cell (p_min_enron); brackets are 95% intervals clustered by target document. Stub results are labelled as such and never pooled with model results. Every red value is filled from the configuration named in Table V (see RESULTS_TODO.md).

A. RQ1: Which artifacts leak, and is it new to the user?

a) Observation.: The per-user cache and the question-and-answer memory hold only what the victim was shown, so their $\mathrm{LM}^{\mathrm{new}}$ is zero by construction; agent memory that keeps tool outputs is an exception (?? VIII-F0b). [R1: LM and LM_new under baseline and which artifact carries them; share of leakage that is re-disclosure; canary-only leak rate as a judge-free check; share of secrets that survive into model-written summaries.]

b) Interpretation.: [Whether each artifact leaks on its own once re-disclosure is separated out. Pilot runs (Section A) found re-disclosure to be most of the stub's leakage (0.527 of 0.585) but under half of Qwen2.5-3B's (0.336 of 0.694): real models answer "remind me" questions with facts the user never asked about.]

TABLE V
Experiments, all on Enron with canaries. Model: L8 = Llama-3.1-8B (4-bit, served locally); stub = deterministic validation harness. Units: revocation events per cell, or test probes for existence.

RQ	Model	Grid	Config	Units
1, 2, 4, 5	L8	preset (baseline, authz_prefilter, epoch, full) × single-document revocation; daily crawl; model-written summaries; probes at 0 s, 1 h, 24 h, including never-asked facts and retained documents	p_min_enron	50
–	L8	same cells, 2 seeds × 1 event; timing check only, not reported	p_min_pilot	2
4, 5	stub	preset (baseline, baseline_shared, authz_prefilter, authz_postfilter, epoch, flush, full) × single-document revocation; extractive summaries; one-sentence answers; probes from 0 s to 48 h	p_min_defenses_stub	200
3	stub	preset (strawman, baseline, baseline_shared, legacy, epoch, full, full_user); insider and outsider; train e1–e3, test e4–e5	p_min_existence_stub	[n]
C1	stub	document edited after revocation; baseline, authz_prefilter, epoch, full	p_min_update_stub	200

TABLE VI
Post-revocation leakage per preset (single-document revocation, daily crawl, 50 events per cell). LM^{new}: secrets the victim was never shown; LM^{re}: re-disclosure; $LM_{\geq 1h}$: from probes at least one hour after revocation. Per-artifact columns are attributed from ground truth and are not exclusive.

Preset	LM	LM^{new}	LM^{re}	$LM_{\geq 1h}$	LM_{index}	LM_{cache}	LM_{memory}
baseline	[]	[]	[]	[]	[]	[]	[]
authz_prefilter	[]	[]	[]	[]	[]	[]	[]
epoch	[]	[]	[]	[]	[]	[]	[]
full	[]	[]	[]	[]	[]	[]	[]

B. RQ2: How long does leakage last?

[R2: per-artifact lifetimes over the probes at 0 s, 1 h and 24 h (Kaplan–Meier, share censored at 24 h) from p_min_enron, and over 0 s–48 h from p_min_defenses_stub for the index under a daily crawl (validated against $\Delta/2$ in Section B) and the cache under a 3600 s TTL.]

C. RQ3: What can an API client infer about hidden documents?

a) Observation.: [R3: AUC per preset, insider vs outsider, per channel; sibling vs singleton targets (most Enron targets are singletons).] In the pilot runs the old baseline, whose cache was shared across the tenant, reached AUC 0.714 [0.664, 0.765] for insiders and 0.512 [0.490, 0.532] for outsiders: almost all of its signal was restricted answers served to insiders from the shared cache (Section A). That is an authorization failure rather than a side channel, which is why baseline now scopes the cache per user and the shared cache is measured as its own preset. The stub's latency is simulated, so this paper makes no claim about timing channels of served models.

D. RQ4: What closes each channel?

a) Defenses compared.: An identifier-based purge (legacy) cannot reach a summary; in the pilot, legacy lowered memory leakage (0.322 to 0.230) but not total leakage, which the stale index dominated. [R4: the published retrieval-time design (authz_prefilter) removes index leakage and leaves cache and memory leakage; epoch and full reach zero; the retained-recall and refusal columns decide between them. State the result whichever way it goes: if epoch invalidation keeps as much retained recall as taint, the simpler mechanism is the recommendation. Stub rows: whether flush, the post-filter design and the shared cache change the picture.]

E. RQ5: What does the defense cost?

[R5: retained-document recall and refusals per preset (model run and stub), and IAM calls and per-document authorization decisions per query, from the revocation runs.]

F. Robustness

a) Post-revocation updates (stub).: [R6: LM^{upd} per preset from p_min_update_stub.]

b) Unmodified open-source stack.: To check that the leaks are not artifacts of our pipeline, we rebuilt the scenario from library defaults only: a LangChain agent (create_agent, LangChain 1.4) with a retrieval tool over Chroma, SummarizationMiddleware for chat memory and a RedisVL SemanticCache (Redis 8) scoped per user with a tag filter [8], [9], generating with Qwen2.5-7B (scripts/real_stack_demo.py). The retrieval tool checks the live identity system on every call, so the index never serves a revoked document. For 10 restricted Enron e-mails, a victim held a five-turn session, lost the e-mail's group, and asked again. In 10/10 cases retrieval refused the e-mail and a fresh session without the cache disclosed nothing. The per-user cache still returned the stored answer and disclosed a corpus-unique value or the canary in 6/10 cases (7/10 with the model judge), which is every case in which the warm-up had shown one; the same

TABLE VII
Existence inference with the stub generator on Enron (combined attacker, with-reranker profile, held-out tenants e4–e5), insiders and outsiders separately. Cells marked [s] are structural: the pre-filter hides the target in both worlds, so only shared state can differ.

Preset	Insider AUC	Outsider AUC	Insider TPR@1%FPR	Strongest channel
strawman	[]	[]	[]	[]
baseline	[]	[]^{s}	[]	[]
baseline_shared	[]	[]^{s}	[]	[]
legacy	[]	[]^{s}	[]	[]
epoch	[]	[]^{s}	[]	[]
full	[]	[]^{s}	[]	[]
full_user	[]	[]^{s}	[]	[]

TABLE VIII
Defenses compared on the same paired events (single-document revocation, daily crawl). Top: model run (50 events per cell). Bottom: stub (200 events per cell, extractive summaries, one-sentence answers) with insider existence AUC from the stub existence run. Retained recall and refusals: answers about documents the victim still holds, in the session where the revocation happened.

Run	Preset	LM	LM^{new}	Retained recall	Refusals	Insider AUC	Decisions/query
L8	baseline	[]	[]	[]	[]	–	0
L8	authz_prefilter	[]	[]	[]	[]	–	[]
L8	epoch	[]	[]	[]	[]	–	[]
L8	full	[]	[]	[]	[]	–	[]
stub	baseline	[]	[]	[]	[]	[]	0
stub	baseline_shared	[]	[]	[]	[]	[]	0
stub	authz_prefilter	[]	[]	[]	[]	–	[]
stub	authz_postfilter	[]	[]	[]	[]	–	[]
stub	epoch	[]	[]	[]	[]	[]	[]
stub	flush	[]	[]	[]	[]	–	[]
stub	full	[]	[]	[]	[]	[]	[]

session disclosed one in 5/10 (6/10). In two of those, no summary had been written yet: the agent's state keeps the retrieved e-mail verbatim as a tool message, so session memory holds the source text itself, not only the answers. Every response is in results/real_stack_demo/demo.json and was checked by hand.

IX. Discussion

a) Revocation is a property of every copy.: [D1: one paragraph from RQ1/RQ2: each artifact heals on its own schedule (crawl period, TTL, session retention), so fixing one propagation path leaves the others open; the retrieval-time design is correct about the artifact it owns.]

b) What a defense for derived state has to buy.: Once read-time checks are in place, zero leakage is cheap: a per-user epoch reaches it with a few lines of code. A finer mechanism has to earn its place on what it lets the user keep. [D2: state the measured retained-recall gap between epoch and full (model run) and between epoch, flush and full (stub), and whether the gap justifies taint.] Finer taint, at the level of summary sentences or of the memory items an answer actually used, needs attribution that real models do not provide; context attribution [89] and permissive information-flow analysis [88] are the candidate routes, and each needs its own soundness argument.

c) Shared state turns re-disclosure into new disclosure.: Re-showing a user what it already saw gives it nothing new, but the same mechanism writes revoked content into state that other users read: in the pilot, a tenant-shared cache served restricted answers to insiders who never had access (Section A). Agent memory that keeps tool outputs goes further for the revoked user itself, since it holds retrieved text the user may never have been shown (?? VIII-F0b).

d) Metadata and content are separate problems.: With a pre-filter, the score and reranker channels carry no signal because the target is invisible in both worlds; what remains is shared state and timing.

e) Practical guidance, limited to what was measured.: Scope semantic caches per user, or check every hit against live authorization. Enforce revocation on read against live identity state rather than waiting for propagation. Bound chat-session retention, and treat tool outputs kept in agent memory as copies of the source documents. Filter by ACL before scoring, and never expose scores of withheld candidates. Budget separately for users who re-type what they already read.

X. Limitations

a) Testbed, not production.: The deployment model follows documented framework and product behaviour (Table II), but every number except the real-stack check (?? VIII-F0b, 10 targets, one model) comes from our pipeline. A deployment with different cache scoping, memory design or sync schedule will see different values; the per-artifact attribution and the lifetime structure are what we expect to transfer.

b) One model, small samples.: The model run uses one model, Llama-3.1-8B in its default 4-bit build, with 50 revocation events per cell; larger and hosted models are not measured, and quantization may change what the model repeats. Intervals are correspondingly wide, and the broader defense comparison, the existence experiment and the update experiment use the stub generator only.

c) Verbatim probes.: Fresh-session probes repeat the warm-up question verbatim, so they hit the cache at any similarity threshold; cache leakage is measured for the repeated question, not for rewordings.

d) Corpus.: The Enron text, questions and answers are real, but tenancy and permissions are derived from each message's recipients, not taken from a real access-control system, and facts are spans of benchmark answers. 457 of the 500 restricted Enron e-mails have no same-thread sibling, so few existence targets have a hard negative in the twin world. The corpus does not have the scale of an enterprise collection.

e) One retrieval stack.: Every run uses the same MiniLM encoder and cross-encoder; cache thresholds are properties of that embedding space.

f) Judging.: Exact matching of corpus-unique values misses paraphrased disclosure. The model judge is not validated against human labels, so we cannot bound how much exact matching misses.

g) Attacker strength.: The existence attacker is gradient boosting on hand-designed features with four fixed probes, trained on shadow tenants. An adaptive attacker that repeats probes, or one with unlimited queries, may do better, so chance-level results are statements about this attacker.

h) Timing.: Time between probes is simulated, and the existence experiment's generation latency comes from the stub's latency model, so its timing results say nothing about a served model.

i) Scope of the defenses.: They cover the index, the cache and chat memory. Logs, exports, fine-tuning data and tool outputs (assumption A2) are out of scope, and no artifact-level mechanism stops a user who paraphrases what it once read (A1).

XI. Conclusion

Revoking access in the identity system of a multi-tenant RAG application does not revoke the copies the application has already made. [C: three to four sentences with the final numbers from configs/min/: how much a revoked user recovers under baseline, how much of it is new, and through which artifact; what the retrieval-time design leaves; which read-time defense reaches zero at the lowest utility cost; insider existence inference with and without the defense (stub).] Re-typed content remains outside what any artifact-level mechanism can follow.

Future work: role, mover and reclassification revocations with bystanders, lost revocation hooks, long sessions, paste-back laundering at scale, larger and hosted models, lifetime sweeps over sync policy, session retention and revocation gap, cache-threshold sweeps with natural rewordings, and leave-one-out ablations of the defense; the testbed already implements each of them.

XII. Ethics

a) Data.: We use the public Enron e-mail corpus [20], released during the Federal Energy Regulatory Commission investigation and widely used in research, and the public EnronQA benchmark [21] built on it. We do not collect new personal data and add nothing that identifies anyone beyond what the public corpus contains. The canary sentences injected into restricted e-mails are random codes that we generated; they contain no real information.

b) Human subjects.: The study involves no human subjects. The only people who read or label model responses are the authors.

c) Real-stack demonstration.: The demonstration in ?? VIII-F0b uses documented, configurable behaviour of open-source libraries (LangChain, Chroma, RedisVL) in a local test deployment on the authors' machine. It touches no third-party systems, accounts or data, and it exploits no bug: the leaks follow from how the application combines the components. [State whether the maintainers of LangChain and RedisVL were notified, and when; or that no notification was made because the behaviour is documented.]

Artifact Availability

The testbed, the corpus generator and the Enron corpus builder, every experiment configuration, the analysis scripts and the per-experiment analysis outputs are available at https://github.com/Mohit-Talgotra/RAGLeakage. The per-query logs ([size]) are deposited at [Zenodo DOI].

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459–9474.

[2] J. Jeong and S.-G. Lee, "Permission-aware RAG: Identity and access management (IAM)-based access filtering in multi-resource environments," IEEE Access, vol. 13, pp. 192 819–192 835, 2025.

[3] B. Chen, J. Tackman, M. Setälä, T. Poranen, and Z. Zhang, "Integrating access control with retrieval-augmented generation: A proof of concept for managing sensitive patient profiles," in Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing (SAC '25), 2025.

[4] H. Zhong, M. Lentz, N. Narodytska, A. Szekeres, and K. Rong, "HONEYBEE: Efficient role-based access control for vector databases via dynamic partitioning," Proceedings of the ACM on Management of Data, vol. 4, no. 1, pp. 1–26, 2026.

[5] Microsoft, "Security filters for trimming results in Azure AI Search," https://learn.microsoft.com/en-us/azure/search/search-security-trimming-for-azure-search, 2026, accessed October 2026.

[6] ——, "Microsoft Graph connectors overview," https://learn.microsoft.com/en-us/graph/connecting-external-content-connectors-overview, 2026, accessed October 2026.
[7] F. Bang, "GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings," in Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), 2023.
[8] Redis, "RedisVL SemanticCache," https://docs.redisvl.com/, 2026, accessed October 2026.
[9] LangChain, "ConversationSummaryBufferMemory (api reference)," https://python.langchain.com/, 2026, accessed October 2026.
[10] LlamaIndex, "Chat memory: ChatSummaryMemoryBuffer," https://docs.llamaindex.ai/, 2026, accessed October 2026.
[11] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv preprint arXiv:2310.08560, 2023.
[12] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," Proceedings of the IEEE, vol. 63, no. 9, pp. 1278–1308, 1975.
[13] Joint Task Force, "Security and privacy controls for information systems and organizations," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-53 Rev. 5, 2020.
[14] S. Büttcher and C. L. A. Clarke, "A security model for full-text file system search in multi-user environments," in 4th USENIX Conference on File and Storage Technologies (FAST 05). USENIX Association, 2005.
[15] N. Gelernter and A. Herzberg, "Cross-site search attacks," in Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS), 2015.
[16] C. Gu, X. L. Li, R. Kuditipudi, P. Liang, and T. Hashimoto, "Auditing prompt caching in language model APIs," in Proceedings of the 42nd International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, vol. 267, 2025, pp. 20 477–20 496.
[17] L. Song, Z. Pang, W. Wang, Z. Wang, X. Wang, H. Chen, W. Song, Y. Jin, D. Meng, and R. Hou, "The early bird catches the leak: Unveiling timing side channels in LLM serving systems," IEEE Transactions on Information Forensics and Security, vol. 20, pp. 11 431–11 446, 2025.
[18] X. Zheng, H. Han, S. Shi, Q. Fang, Z. Du, X. Hu, and Q. Guo, "InputSnatch: Stealing input in LLM services via timing side-channel attacks," arXiv preprint arXiv:2411.18191, 2024.
[19] N. Carlini and M. Nasr, "Remote timing attacks on efficient language model inference," arXiv preprint arXiv:2410.17175, 2024.
[20] B. Klimt and Y. Yang, "The Enron corpus: A new dataset for email classification research," in Machine Learning: ECML 2004, ser. Lecture Notes in Computer Science, vol. 3201. Springer, 2004, pp. 217–226.
[21] M. J. Ryan, D. Xu, C. Nivera, and D. Campos, "EnronQA: Towards personalized RAG over private documents," 2025.
[22] N. A. Obuchowski, "Nonparametric analysis of clustered ROC curve data," Biometrics, vol. 53, no. 2, pp. 567–578, 1997.
[23] A. C. Myers and B. Liskov, "A decentralized model for information flow control," in Proceedings of the Sixteenth ACM Symposium on Operating Systems Principles (SOSP). ACM, 1997, pp. 129–142.
[24] E. Elnikety, A. Mehta, A. Vahldiek-Oberwagner, D. Garg, and P. Druschel, "Thoth: Comprehensive policy compliance in data retrieval systems," in 25th USENIX Security Symposium (USENIX Security 16). USENIX Association, 2016, pp. 637–654.
[25] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," in Advances in Neural Information Processing Systems, 2026.
[26] R. Pang, R. Cáceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, and M. Wang, "Zanzibar: Google's consistent, global authorization system," in 2019 USENIX Annual Technical Conference (USENIX ATC 19). USENIX Association, 2019, pp. 33–46.
[27] A. Mehta, E. Elnikety, K. Harvey, D. Garg, and P. Druschel, "Qapla: Policy compliance for database-backed systems," in 26th USENIX Security Symposium (USENIX Security 17). USENIX Association, 2017.
[28] K. Cohn-Gordon, G. Damaskinos, D. Neto, J. Cordova, B. Reitz, B. Strahs, D. Obenshain, P. Pearce, and I. Papagiannis, "DELF: Safeguarding deletion correctness in online social networks," in 29th USENIX Security Symposium (USENIX Security 20). USENIX Association, 2020.
[29] K. D. Albab, I. Sharma, J. Adam, B. Kilimnik, A. Jeyaraj, R. Paul, A. Agvanian, L. Spiegelberg, and M. Schwarzkopf, "K9db: Privacy-compliant storage for web applications by construction," in 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2023.
[30] L. Tavakoli and M. Sanderson, "Deletion isn't enough: Auditing RAG for selective forgetting," in Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26), 2026.
[31] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," arXiv preprint arXiv:2503.18813, 2025.
[32] B. Wang et al., "Unveiling privacy risks in LLM agent memory," in Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), 2025.
[33] J. Crampton, W. Leung, and K. Beznosov, "The secondary and approximate authorization model and its application to Bell-LaPadula policies," in Proceedings of the 11th ACM Symposium on Access Control Models and Technologies (SACMAT), 2006.
[34] Q. Wei, M. Ripeanu, and K. Beznosov, "Cooperative secondary authorization recycling," IEEE Transactions on Parallel and Distributed Systems, vol. 20, no. 2, 2009.
[35] Y. Liu, W. Tome, L. Zhang, D. Choffnes, D. Levin, B. Maggs, A. Mislove, A. Schulman, and C. Wilson, "An end-to-end measurement of certificate revocation in the web's PKI," in Proceedings of the 2015 Internet Measurement Conference (IMC), 2015.
[36] P. Bailey, D. Hawking, and B. Matson, "Secure search in enterprise webs: Tradeoffs in efficient implementation for document level security," in Proceedings of the 15th ACM International Conference on Information and Knowledge Management (CIKM), 2006, pp. 493–502.
[37] S. Rizvi, A. Mendelzon, S. Sudarshan, and P. Roy, "Extending query rewriting techniques for fine-grained access control," in Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data, 2004.
[38] Qdrant, "Filtering," https://qdrant.tech/documentation/concepts/filtering/, 2026, accessed October 2026.
[39] Pinecone, "Filter by metadata," https://docs.pinecone.io/, 2026, accessed October 2026.
[40] Chroma, "Metadata filtering," https://docs.trychroma.com/, 2026, accessed October 2026.
[41] S. Gollapudi, N. Karia, V. Sivashankar, R. Krishnaswamy, N. Begwani, S. Raz, Y. Lin, Y. Zhang, N. Mahapatro, P. Srinivasan, A. Singh, and H. V. Simhadri, "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in Proceedings of the ACM Web Conference 2023 (WWW), 2023.
[42] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," Proceedings of the ACM on Management of Data, vol. 2, no. 3, 2024.
[43] A. Gupta and I. S. Mumick, "Maintenance of materialized views: Problems, techniques, and applications," IEEE Data Engineering Bulletin, vol. 18, no. 2, 1995.
[44] D. D. Redell and R. S. Fabry, "Selective revocation of capabilities," in Proceedings of the International Workshop on Protection in Operating Systems, 1974.
[45] M. Kallahalla, E. Riedel, R. Swaminathan, Q. Wang, and K. Fu, "Plutus: Scalable secure file sharing on untrusted storage," in 2nd USENIX Conference on File and Storage Technologies (FAST), 2003.

[46] M. Backes, C. Cachin, and A. Oprea, "Lazy revocation in cryptographic file systems," in Third IEEE International Security in Storage Workshop (SISW), 2005.

[47] Y. Cao and J. Yang, "Towards making systems forget with machine unlearning," in 2015 IEEE Symposium on Security and Privacy (SP), 2015, pp. 463–480.

[48] S. Garg, S. Goldwasser, and P. N. Vasudevan, "Formalizing data deletion in the context of the right to be forgotten," in Advances in Cryptology – EUROCRYPT 2020, 2020.

[49] L. Tsai, H. Gross, E. Kohler, F. Kaashoek, and M. Schwarzkopf, "Edna: Disguising and revealing user data in web applications," in Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), 2023.

[50] S. Shastri, V. Banakar, M. Wasserman, A. Kumar, and V. Chidambaram, "Understanding and benchmarking the impact of GDPR on database systems," Proceedings of the VLDB Endowment, vol. 13, no. 7, 2020.

[51] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 141–159.

[52] S. Wang et al., "When machine unlearning meets retrieval-augmented generation (RAG): Keep secret or forget knowledge?" arXiv preprint arXiv:2410.15267, 2024.

[53] Q. Wang, L. Ding, Y. Cao, Z. Tian, S. Wang, D. Tao, and L. Guo, "Recursively summarizing enables long-term dialogue memory in large language models," arXiv preprint arXiv:2308.15022, 2023.

[54] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23), 2023.

[55] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, "MemoryBank: Enhancing large language models with long-term memory," in Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, 2024.

[56] S. Chen, R. Wang, X. Wang, and K. Zhang, "Side-channel leaks in web applications: A reality today, a challenge tomorrow," in 2010 IEEE Symposium on Security and Privacy, 2010.

[57] A. Bortz and D. Boneh, "Exposing private information by timing web applications," in Proceedings of the 16th International Conference on World Wide Web (WWW), 2007, pp. 621–628.

[58] E. W. Felten and M. A. Schneider, "Timing attacks on web privacy," in Proceedings of the 7th ACM Conference on Computer and Communications Security (CCS), 2000, pp. 25–32.

[59] G. Wu, Z. Zhang, Y. Zhang, W. Wang, J. Niu, Y. Wu, and Y. Zhang, "I know what you asked: Prompt leakage via KV-cache sharing in multi-tenant LLM serving," in Network and Distributed System Security Symposium (NDSS), 2025.

[60] R. Weiss, D. Ayzenshteyn, G. Amit, and Y. Mirsky, "What was your prompt? a remote keylogging attack on AI assistants," in 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, 2024, pp. 3367–3384.

[61] Z. Zhang, Z. Liu, Y. Xie, Q. Huang, and D. She, "From similarity to vulnerability: Key collision attack on LLM semantic caching," in Proceedings of the 43rd International Conference on Machine Learning, 2026.

[62] M. Afiffy, M. W. Fakhr, and F. A. Maghraby, "Enhancing adversarial resilience in semantic caching for secure retrieval augmented generation systems," Scientific Reports, vol. 16, p. 5936, 2026.

[63] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar, "MeanCache: User-centric semantic caching for LLM web services," arXiv preprint arXiv:2403.02694, 2024.

[64] M. Anderson, G. Amit, and A. Goldsteen, "Is my data in your retrieval database? membership inference attacks against retrieval augmented generation," in Proceedings of the 11th International Conference on Information Systems Security and Privacy (ICISSP). SciTePress, 2025, pp. 474–485.

[65] Y. Li, G. Liu, Y. Yang, and C. Wang, "Generating is believing: Membership inference attacks against retrieval-augmented generation," arXiv preprint arXiv:2406.19234, 2024.

[66] A. Naseh, Y. Peng, A. Suri, H. Chaudhari, A. Oprea, and A. Houmansadr, "Riddle me this! Stealthy membership inference for retrieval-augmented generation," arXiv preprint arXiv:2502.00306, 2025.

[67] M. Liu, S. Zhang, and C. Long, "Mask-based membership inference attacks for retrieval-augmented generation," in Proceedings of the ACM on Web Conference 2025 (WWW '25), 2025, pp. 2894–2907.

[68] S. Zeng, J. Zhang, P. He, Y. Xing, Y. Liu, H. Xu, J. Ren, S. Wang, D. Yin, Y. Chang, and J. Tang, "The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG)," in Findings of the Association for Computational Linguistics: ACL 2024, 2024, pp. 4505–4524.

[69] Z. Qi, H. Zhang, E. P. Xing, S. Kakade, and H. Lakkaraju, "Follow my instruction and spill the beans: Scalable data extraction from retrieval-augmented generation systems," in International Conference on Learning Representations (ICLR), 2025, pp. 48 733–48 755.

[70] C. Di Maio, C. Cosci, M. Maggini, V. Poggioni, and S. Melacci, "Pirates of the RAG: Adaptively attacking LLMs to leak knowledge bases," arXiv preprint arXiv:2412.18295, 2024.

[71] J. X. Morris, V. Kuleshov, V. Shmatikov, and A. M. Rush, "Text embeddings reveal (almost) as much as text," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[72] T. Koga, R. Wu, and K. Chaudhuri, "Privacy-preserving retrieval-augmented generation with differential privacy," arXiv preprint arXiv:2412.04697, 2024.

[73] S. Zeng, J. Zhang, P. He, J. Ren, T. Zheng, H. Lu, H. Xu, H. Liu, Y. Xing, and J. Tang, "Mitigating the privacy issues in retrieval-augmented generation (RAG) via pure synthetic data," arXiv preprint arXiv:2406.14773, 2024.

[74] N. Carlini, S. Chien, M. Nasr, S. Song, A. Terzis, and F. Tramèr, "Membership inference attacks from first principles," in 2022 IEEE Symposium on Security and Privacy (SP), 2022, pp. 1897–1914.

[75] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," IEEE Journal on Selected Areas in Communications, vol. 21, no. 1, pp. 5–19, 2003.

[76] N. Zeldovich, S. Boyd-Wickizer, E. Kohler, and D. Mazières, "Making information flow explicit in HiStar," in 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI 06). USENIX Association, 2006, pp. 263–278.

[77] M. Krohn, A. Yip, M. Brodsky, N. Cliffer, M. F. Kaashoek, E. Kohler, and R. Morris, "Information flow control for standard OS abstractions," in Proceedings of the Twenty-First ACM SIGOPS Symposium on Operating Systems Principles (SOSP). ACM, 2007, pp. 321–334.

[78] A. Yip, X. Wang, N. Zeldovich, and M. F. Kaashoek, "Improving application security with data flow assertions," in Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (SOSP). ACM, 2009, pp. 291–304.

[79] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones," in 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10). USENIX Association, 2010.

[80] J. Park, D. Nguyen, and R. Sandhu, "A provenance-based access control model," in Tenth Annual International Conference on Privacy, Security and Trust (PST), 2012.

[81] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in Database Theory — ICDT 2001, ser. Lecture Notes in Computer Science, vol. 1973. Springer, 2001, pp. 316–330.

[82] F. Wu, E. Cecchetti, and C. Xiao, "System-level defense against indirect prompt injection attacks: An information flow control perspective," arXiv preprint arXiv:2409.19091, 2024.

[83] P. Y. Zhong, S. Chen, R. Wang, M. McCall, B. L. Titzer, H. Miller, and P. B. Gibbons, "RTBAS: Defending LLM agents against prompt injection and privacy leakage," arXiv preprint arXiv:2502.08966, 2025.

[84] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "IsolateGPT: An execution isolation architecture for LLM-based

agentic systems," in Network and Distributed System Security Symposium (NDSS), 2025.
[85] E. Bagdasarian, R. Yi, S. Ghalebikesabi, P. Kairouz, M. Gruteser, S. Oh, B. Balle, and D. Ramage, "AirGapAgent: Protecting privacy-conscious conversational agents," in Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS), 2024.
[86] T. Tiwari, S. Gururangan, C. Guo, W. Hua, S. Kariyappa, U. Gupta, W. Xiong, K. Maeng, H.-H. S. Lee, and G. E. Suh, "Information flow control in machine learning through modular model architecture," in 33rd USENIX Security Symposium, 2024.
[87] B. Jayaraman, V. J. Marathe, H. Mozaffari, W. F. Shen, and K. Kenthapadi, "Permissioned LLMs: Enforcing access control in large language models," arXiv preprint, 2025.
[88] S. A. Siddiqui, R. Gaonkar, B. Köpf, D. Krueger, A. Paverd, A. Salem, S. Tople, L. Wutschitz, M. Xia, and S. Zanella-Béguelin, "Permissive information-flow analysis for large language models," arXiv preprint arXiv:2410.03055, 2024.
[89] B. Cohen-Wang, H. Shah, K. Georgiev, and A. Madry, "ContextCite: Attributing model generation to context," in Advances in Neural Information Processing Systems, vol. 37, 2024.
[90] Zilliz, "GPTCache: semantic cache for LLMs," https://github.com/zilliztech/GPTCache, 2026, accessed October 2026.
[91] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in 2017 IEEE Symposium on Security and Privacy (SP), 2017, pp. 3–18.
[92] N. Carlini, C. Liu, Ú. Erlingsson, J. Kos, and D. Song, "The secret sharer: Evaluating and testing unintended memorization in neural networks," in 28th USENIX Security Symposium (USENIX Security 19), 2019, pp. 267–284.
[93] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), 2019, pp. 3982–3992.
[94] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 5776–5788.
[95] E. L. Kaplan and P. Meier, "Nonparametric estimation from incomplete observations," Journal of the American Statistical Association, vol. 53, no. 282, pp. 457–481, 1958.
[96] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.
[97] S. Holm, "A simple sequentially rejective multiple test procedure," Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65–70, 1979.

Appendix A
Pilot study (logs of 7–12 October 2026)

This appendix reports a pilot, not the paper's results. Before the final runs we re-analysed the logs of an earlier pilot (one generated corpus of 500 documents, 51 words per restricted document, one chunk each; stub, Qwen2.5-3B and Llama-3.1-8B; the deployment's cache shared across each tenant) with the judge and statistics of Section VI. Four results changed the design.

a) The judge.: The pilot's analysis, which excluded a value only if it appeared in the response's own source documents, reported one leaking event under the full defense with Llama-3.1-8B and one with the stub. Both were values shared with other documents: a city name that the model repeated while saying the document did not mention it, and a hazard ratio that another project's report also contained. With corpus-unique judging, the full defense leaks 0.000 in all 100 events with each generator. 21% of the pilot corpus's fact values and 43% of Enron's recur in another document.

b) Most stub leakage is re-disclosure.: Under the pilot's baseline, the stub disclosed LM = 0.585 [0.526, 0.641] (250 events, 124 targets), of which $\mathrm{LM}^{\mathrm{new}} = 0.058$ [0.025, 0.098] was new to the victim; the warm-up had already shown it 66% of the target's secrets. Real models differ: Qwen2.5-3B disclosed 0.694 [0.649, 0.733] with $\mathrm{LM}^{\mathrm{new}} = 0.358$ [0.306, 0.409], and Llama-3.1-8B 0.506 [0.458, 0.551] with $\mathrm{LM}^{\mathrm{new}} = 0.145$ [0.109, 0.183] (100 events each). This motivated the decomposition, the probe about unseen facts, the post-revocation update, and real models as primary evidence.

c) The existence signal of the pilot baseline was a shared cache.: Insiders reached AUC 0.714 [0.664, 0.765] and outsiders 0.512 [0.490, 0.532]; the content channel alone gave insiders 0.717. Cached answers generated for an authorized colleague were served to insiders without access. With the full defense the combined AUC was 0.495 [0.486, 0.504] with the stub and 0.540 [0.520, 0.560] with Llama-3.1-8B, whose latency was replayed from a generation cache; the existence results of this paper use the stub.

d) Identifier purges and the stale index.: The pilot's legacy preset matched baseline in total leakage (0.585 against 0.585) because both keep the stale index, which dominates, while legacy did lower memory leakage (0.230 against 0.322). Total leakage therefore cannot show that identifier purges miss summaries; an ablation with the live ACL check in place would (future work).

Appendix B
Simulator validation

[V: index-leak lifetime against the closed form $\Delta/2$ for the daily crawl, and cache-leak lifetime against the TTL, from p_min_defenses_stub. These hold by construction and validate the simulated clock; they are not findings.]

Appendix C
Stub results

[S: per-artifact leakage of p_min_defenses_stub (one-sentence answers, extractive summaries) as an upper bound on what summaries retain; differences from the model run.]